

Metadata:

Assignment 2: Ciphers and Encryption

Wallace Wei (wzw4)

Main.java

Directions to run code:

Run Main first to built the out file

Use jar, choose the *build artifacts* option from the menu item *Build* to create the CLI. Or, you can use in the file:

```
javac -d out src/cipher/*.java
```

```
jar cfm SimpleCLI.jar MANIFEST.MF -C out .
```

Then run it

Read the a2 doc for commands

Summary:

One of the most challenging parts of this assignment was reading the A2 document correctly, and understanding what it meant. Also, writing comments, properly throwing exceptions was difficult as well, along with shortening code, and small bugs were all great challenges.

Another major challenging part of the assignment was Chunking and RSA, especially when inputting and outputting the correct encryptions/decryptions, it's impossible to verify the multi-byte data thus very hard to debug. Plus, chunking multiple times caused me issues and took me a while to fix.

Other noteworthy design/implementation decisions I made are summarized below:

– I used a “decoder” array for ***Alphabetic Ciphers*** to allow for easy decryption.

- I move multi-byte characters to the next Chunk if there wasn't enough space left
- Monoalphabetic Cipher must be initialized with a permutation of the 26 alphabetic letters as a string, or there is an error
- Decrypting ciphertext also enforces the string to be lowercase, and alphabetic or whitespace
- In RSA, errors will be thrown in decryption for the following cases:
 - Header byte (size) is larger than chunk length, or smaller than the amount of non-zero bytes
 - The bytes are too invalid for a character (tries to handle it by using question mark characters if possible)
 - (but, if the header byte is larger than the amount of non-zero bytes, it is left as is)
- These are meant to guard against jumbled encryption byte code

Major Issues:

- Dividing up classes to maximize shared code, and how to use inheritance effectively;
- RSA using UTF-8, not ASCII;
- Testing the Command Line Interface properly.

Specification Changes:

- If an invalid alphabet is given to the ***Monoalphabetic Cipher*** [not a permutation string (no duplicates) of only the alphabet], then an exception is thrown.

“abcdefghijklmnopqrstuvwxyz” = valid

“zzbcdefghijklmnopqrstuvwxyz” = invalid

“zbcdefghijklmnopqrstuvwxyz” = invalid

“abcdefghijklmnopqrstuvwxyz ” = invalid

Etcetera

- If there is one byte left in a chunk, and you have to read in a 4-byte emoji for example, the chunk is padded out and the emoji is left for the next chunk. This is needed because the emoji needs to be read in full to be properly converted to its corresponding character form.
- Empty **Vigenere Cipher** String (after removing non-alphabetic) in the Constructor results in an exception, because Vigenere needs letters to work as a cipher.
- Decrypted Strings are enforced to be lowercase, alphabetic, and keep whitespace because it is to let a user input into the CLI something like “String”, and will not force them to input **String** without quotation marks
- Doesn't enforce invalid byte lengths for modulus, or invalid private or public keys. I believe the user should be able to input small primes if they would like to test the system, and checking private or public keys is virtually impossible because it requires factoring, which is at the heart of RSA's difficulty
- The string encrypt and decrypt methods that use cipher/plaintext are null
- When decrypting, invalid chunk sizes, invalid characters, or incompatible header length (too large) with the bytes will result in exceptions (note that if somehow the header length is too big and includes null bytes, that is acceptable)

This is done to root out any bad encryptions quickly

Exceptions thrown:

ChunkSize > 126

A bunch of invalid characters, such that it can't understand it at all to convert to UTF8

Header says there are 28 bytes, actually 30 bytes

Design and Implementation:

The **Main** contains the code for the command line interface.

The key interfaces in this code are the **ChunksReader** and **Cipher**.

ChunksReader is a template for the implementation of the way RSA reads data, with byte chunks of a certain size that are read into, which are then processed with the RSA algorithm chunk by chunk.

Cipher is the basis for all the ciphers in this assignment, as it gives a way for all the ciphers to encrypt/decrypt a message from input to output, or if it is alphabetic, directly, and to save the “key” that makes the cipher tick to a file.

Jar, .MF compiles program for Command Line Interface

(Copy and Paste from the Prelim Design Document for the general design)

For the type hierarchy for the program, I am disclosing **Main** and **Cipher Factory**, as those two classes are isolated by themselves, and are relatively self-explanatory, as **Main** runs all the code and serves as a way for users to interact with the program through the command line, and **Cipher Factory** is a Factory class that allows for access to creating objects of the classes.

The **cipher** is the interface that all the ciphers (Caesar, Random, Vigenere, and RSA) will use.

Chunks implements **ChunksReader**, and does so by using the **BufferedInputStream**, and tries to set a specific **ChunkSize** in bytes, then uses **hasNext** continually in the next chunk to ensure that there is a next byte to use, and takes the byte from the input stream and puts it into the byte array, until it hits the limit.

From this, **RSA** will be its class, as it shares relatively fewer parts than the rest of the classes, as reading information into RSA requires the **ChunkReader**, and the manual encryption and decryption processes are not very similar to the other ones, as this one doesn't rely on the alphabet, but the RSA Algorithm. Furthermore, writing out the encrypted and decrypted messages is vastly different from the other three cases, as RSA uses the **ChunkReader** rather than **Scanner** and breaks messages up into a specific amount of bytes and then writes them, rather than writing each one out line by line regardless of bytes. On top of this, the **save** function is different significantly, as RSA uses three variables to print out that are specific to itself. This is why it is best to use RSA as a class that implements **Cipher** alone. Additionally, it can use any symbol and recognize it in **UTF-8**, unlike the other ones.

The abstract class **AbstractCipher** is used because the **Vigenere**, **Caesar**, and **Random Ciphers** are all alphabetic ciphers, and share common functionalities, like using an alphabet, using a similar encryption and decryption input/output function as they all use **Scanner** to read each line, and apply the cipher or decrypt it and then put it back into an output stream, and use a similar save function, as they take the cipher “key” whether it be a change of the alphabet or the **Vigenere cipher**, and saves this key to a file, changing the wording around for the **Vigenere cipher** from mono to itself.

The **MonoCipher** is a normal class because it can be used by itself, when the user gives the proper alphabet, and has its way of encrypting and decrypting based on this random alphabet given to it.

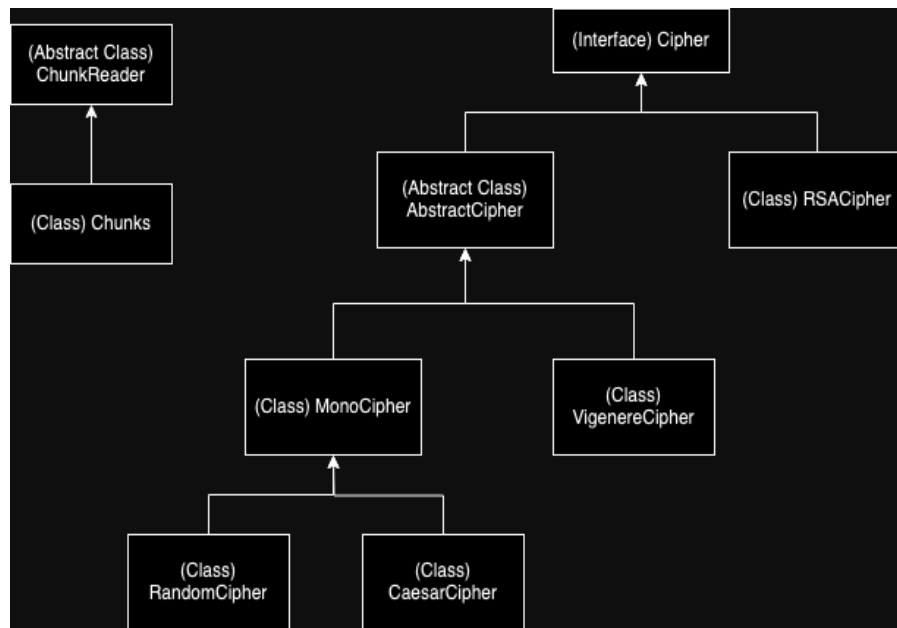


Fig. 1. Class diagrams of Assignment 2.

Finally, the **Caesar** and **Random Ciphers** both use the encryption and decryption of the **MonoCipher**, as well as all the other classes it inherits, but both of them just have different initializers of the alphabet, whether it be shifts, or randomly generated.

I used an ArrayList in the **MonoAlphabet** class order to facilitate the ability to use Collections and the random list feature, which could generate the random alphabet. Plus, it doesn't impede the Caesar Cipher at all.

When trying to "remember" the map from one alphabet to another, an intermediary array is used to speed it up instead of always iterating, which I call a "decoder" array. Basically, it has its indices as the ciphertext letter, and its context is the corresponding plaintext letter.

If $b \rightarrow d$ in ciphertext, $\text{decoder}[1] = 3$

The only real and complicated algorithm I used was RSA, and I got it from the Wikipedia article linked in the design doc. Basically, it takes in data chunk by chunk to encrypt/decrypt them with modular arithmetic as detailed in the design doc.

Also, I used a set, in order to simplify the code for checking a "unique" a-z alphabet, which shortened the implementing time, rather than using an array of 26 zeroes to ensure an a-z alphabet, I used the intrinsic properties of a set (no duplicates).

Some tradeoffs I had to make were whether to make the code shorter, like by packing it into helper methods, or to keep it longer, but more readable/understandable, which I chose the latter.

Also, I had to make the tradeoff of using package code for many RSA methods over the code I already wrote, in order to ensure readability.

I didn't really have much of a strategy in mind, I mostly just coded the hardest things I found first, then built the surrounding features around the "meat and potatoes" of the ciphers, which is more of a bottom-up approach.

I did use a factory pattern, but that was required, so I'll skip it.

As for the user, they can use the command line interface, which acts as a way to input their desired commands. They can manually also call the functions, but that seems rather unnecessary to detail, as they shouldn't do that.

When reading the A2 document, I got tripped on the fact that I had to use UTF-8, not ASCII, and that I had to output byte code for encryption, which wasted some of my time, as it didn't have those features.

Shortening code was hard, as I did most of RSA manually at first, before realizing that pretty much all the things I did manually had a package for it like charsets. Also, I would debug for long times, as I had trouble with hasNext(), as I switched the order of hasNext() and the chunkSize clause for my while loop, which meant that I was processing hasNext() for no reason, and made my code off by 1. This also made me learn that reading the byte code was pretty much useless.

Class Invariants (From Code):

RSA:

```
// modulus is a composite number made of two primes, with bitlength between
1017 and 1023
// public key is a fixed prime, and private key is the modular inverse of the
fixed prime under euler's totient (p-1)(q-1)
// Information about the String is broken up into chunks (126) of hexadecimal
(representing bytes) padded with zeroes
// Byte Code is in chunks of 128, to be read back into and decrypted
// Conforms to the UTF-8 Standard of Character Encoding
```

Vigenere:

```
// Class invariant: the Vigenere Cipher is represented by a constant key, an is
a nonempty string where the letters represent shifts (a=1, b=2 ...)
// This key is cyclically used to encode/decode strings,
```

Rando:

```
// Uses a list to shuffle the alphabet randomly, then records the shuffled
alphabet to a string.
// Also, it updates an array such that the index is the letter used in the
random cipher, and the contents is the actual letter
// This makes it easier to reverse an encrypted random cipher
```

MonoAlphabet:

```
//Class Invariants: newAlphabet is a permutation of the alphabet (26 letters)
with no duplicates
// newAlphabet and decoder are inverses of each other (switching the places of
the letters from index to value, and parameterizing them into numbers for
decoder)
// output is always lowercase alphabetical characters with whitespace
```

Chunks:

```
// Class Invariant: All characters must have their full byte representation put
inside a chunk
// A chunk must <= chunk size 126 (positive integer)
// Given a proper and correct inputstream and valid byte array
// 0 index is the length of byte data, the rest is the byte data
```

Caesar:

```
// Class Invariant: Integer shifts must be made between 0-25
// alphabetString/newAlphabet is the same as the normal alphabet, but
cyclically shifted
// decoder maps from ciphertext to plaintext using indices as ciphertext to
values as plaintext
```

AbstractCipher:

```
// Class Invariants: alphabetString is a valid permutation of letters of the
alphabet
// cipherType represents the type of cipher
// alphabet must stay as an alphabet from a-z in that order
```

Testing:

Unit/Integration Testing

(I tested some of these in my own testing file, some of them I tested by myself through command line)

- Test all the pre-given tests in the BasicCipherTests
- Test the pre-given RSA tests (1-3)
- Test the command line inputs in the design doc
- Test an empty string for each encryption method for all the ciphers, and decrypt that empty string
- Test negative and positive integers over 26 or -26 for a Caesar Cipher to ensure that
- Test the save method (see if it works as intended)
- Test a normal string for all the alphabetic ciphers to see if it preserves whitespace
- Test a string with unallowed characters (non-whitespace, non-alphabetic) for a Caesar Cipher, Viginerie, Random Cipher
- Test the RSA initializer that creates new RSA Keys
- Test the MonoAlphabet with a normal alphabet permutation
- Test the MonoAlphabet with an alphabet too long/short (exc)
- Test the MonoAlphabet with excess characters, even if it has an alphabet permutation without the extra characters (exc)
- Test the Mono Alphabet with a non-unique alphabet (exc)
- Test RSA with a normal string (ASCII)
- Test RSA with only normal multi-character bytes like emojis
- Test RSA with multiple chunks of UTF 8 with different byte-length characters
- Test Exceptions that I raise in the Main class
- Test invalid byte array input for RSA decrypt (Handles it gracefully, just skips random invalid char)

For all of these, I have so far found that all of my code is working as expected after all these reasonably rigorous test cases. I may be missing some test cases, though, especially concerning test cases relating to all the exceptions I raise, which is too hard to exhaust.

I think a lot of these test cases are exhaustive because they form a lot of edge cases to test, and form a way to ensure that all user input can be validated.

Known Problems:

The exceptions thrown in my programs may not exhaust all possibilities.

Documentation and comments might be light.

The code, especially the “Chunks” class, and “RSA” may not be that easy to understand.

The main CLI may be faulty, I didn't get around to fully testing it.

Comments:

I don't exactly count how much time I spent on the assignment, but I would probably spend more than average, as my other classes are pretty light and I enjoy coding and debugging and trying different means to improve the readability and conciseness of this assignment.

The majority of my time was spent debugging and rewriting code for possible improvement in logic and/or conciseness.

The most surprising part of the assignment was how hard it is to process the chunks, and how hard it was to validate multiple-byte characters as it is impossible to make sense of the contents for correctness.

I would suggest to change this assignment by specifying clearly that there are multiple byte characters up front. I have to rewrite a big chunk of my system from scratch when I figured out we are not dealing with ASCII characters only.